

What is injection token?

The Problem: Why do we need Injection Tokens?

Angular's DI framework usually uses TypeScript **classes** as lookup tokens. Because classes exist at runtime, Angular can easily match a dependency request to its provider:

```
TypeScript
// Angular knows exactly what to inject because LoggerService exists at
runtime
constructor(private logger: LoggerService) {}
```

However, TypeScript **interfaces** and **types** are entirely stripped away during compilation. They do not exist at JavaScript runtime. If you try to use an interface as a DI token, Angular has nothing to look up, and the application will fail.

This is where `InjectionToken` comes in. It creates a physical, runtime object that acts as a placeholder or "hook" for the dependency you want to inject.

How to Use an Injection Token

Using an injection token involves three simple steps: **Creating** it, **Providing** it, and **Injecting** it.

1. Define the Token

First, you create an instance of `InjectionToken`. You pass it a description (for debugging purposes) and optionally specify its type.

```
TypeScript
import { InjectionToken } from '@angular/core';

// Define an interface for the shape of the data
export interface AppConfig {
  apiUrl: string;
  theme: 'light' | 'dark';
}

// Create the token
export const APP_CONFIG = new InjectionToken<AppConfig>('app.config');
```

2. Provide the Token

Next, you need to tell Angular's DI container what value should be associated with this token. You do this in your module, component, or environment providers list using the `useValue` (or `useFactory`, `useClass`) property.

```
TypeScript
import { bootstrapApplication } from '@angular/platform-browser';
import { APP_CONFIG } from './config.token';

bootstrapApplication(AppComponent, {
  providers: [
    {

```

What is injection token?

```
        provide: APP_CONFIG,  
        useValue: { apiUrl: 'https://api.example.com', theme: 'dark' }  
    }  
  ]  
});
```

3. Inject the Token

Finally, because the token isn't a class, you cannot just declare it as a type in your constructor. You must use the `@Inject()` decorator (or the `inject()` function) to fetch it.

Option A: Using the `inject()` function (Modern Angular)

TypeScript

```
import { Component, inject } from '@angular/core';  
import { APP_CONFIG } from '../config.token';  
  
@Component({ ... })  
export class AppComponent {  
  // Clear, type-safe, and concise  
  private config = inject(APP_CONFIG);  
  
  constructor() {  
    console.log(this.config.apiUrl); // 'https://api.example.com'  
  }  
}
```

Option B: Using the `@Inject()` Decorator (Classic)

TypeScript

```
import { Component, Inject } from '@angular/core';  
import { APP_CONFIG, AppConfig } from '../config.token';  
  
@Component({ ... })  
export class AppComponent {  
  constructor(@Inject(APP_CONFIG) private config: AppConfig) {  
    console.log(this.config.apiUrl);  
  }  
}
```

Tree-shakable Injection Tokens

If you want to provide a default value for your token without forcing the user to explicitly register it in a providers list, you can make it **tree-shakable**. This is highly recommended for building reusable libraries.

TypeScript

```
export const API_URL = new InjectionToken<string>('api.url', {  
  providedIn: 'root',  
  factory: () => 'https://default-api.com' // Default value factory  
});
```

With this setup, you can immediately `inject(API_URL)` anywhere in your app without adding it to a providers array. If your app never actually uses `API_URL`, Angular's build tools will completely remove (tree-shake) it from the final bundle.

What is injection token?

Summary of Use Cases

You should reach for an `InjectionToken` whenever you need to inject:

- **Configuration Objects:** App settings, environment flags, or API endpoints.
- **Plain Primitives:** Strings, numbers, or booleans (e.g., a library version number).
- **Interfaces:** When implementing a pluggable architecture where multiple classes implement the same interface.
- **Browser APIs:** Wrapping global objects like `window`, `document`, or `localStorage` to make your code safely testable.